Component A Deliverables

1. Interview scripts with your own notes from the staff interview

Interview Notes

- Review documents, make comparisons between 2 documents to see if they are the same to see if they are the same
- Send out email
- Course syllabus for course petitions
- 90% to be the same to pass
    - Learning outcomes, keywords, semantics
    - Topics covered, deliverables
    - if super clear, no send to instructors
- Project based vs Textbased
- Generally don't allow external supplement to support course similarities
- Students have to fill out the form (including email)
- Most frustrated - having to email each student for follow up (Catch this)
    - If syllabus isnt in english or self translated or formatted differently
    - If cant open document
    - Empty submission
    - Reviewing every syllabus
- Students submit documents individually, need a way to put it together (same amount of documents each time)
- Does initial review
- LOOK at course petition page on canvas
- Pick out the students that need to be reviewed to the teacher in a spreadsheet which they review
    - Jason does weed out the super clear ones
- have used AI in the past, but not having leveraged other AI, not as reliable for nuances
- want a system where it summarizes the similarities and then flag areas where there are contention
- Class levels matter as well (400 or above equivalent)
- If students are eligible for a lot of different classes, they talk 1 on 1 with Jason/instructor to decide
- In English? meet the requirements, can i open it?
- instructor has final decision
- Want syllabus/transcript that are approved by UW or a sign off by the instructor previously
- At least 2 weeks to review by instructors
- usually pdf -> 🙂, png, jpeg -> bad 🙁

- keep for 5 years
- spreadsheet notes for student meetings
- Submit up to 3 syllabus for 1 class
- Need to review feedback from instructors before sending it off again
- Constraints
    - Might have UW regulations need to research
    - Can’t use AI to make decisions of transcripts/syllabus
- AI chatbot to help guide it

2. Problem statement and flowchart

When students submit course petitions, staff currently manage the entire process manually, filtering out unreadable or incomplete submissions, emailing each student individually for follow-up, and consolidating documents across a slew of email, Canvas, and spreadsheets, which causes significant time loss on administrative overhead, delayed reviews, and a backlog that makes it harder to give each petition the careful evaluation it deserves.

Petition Review Process

Student syllabus submission workflow

Student submits petition
Up to 3 syllabi per class

↓

Staff: initial triage
Can it be opened? Is it in English?

Pain: bad submissions
Unreadable, non-English, empty, or wrong format

↓

Staff emails student for follow-up
One email per issue, per student

Pain: biggest time sink
Manual follow-up emails are most frustrating step

↓

Consolidate documents
Gather all syllabi per student

↓

Staff reviews syllabus similarity
90% match: outcomes, topics, level

Pain: manual review
Every syllabus read by hand; no structured summary

↓

Weed out clear approvals
Jason flags edge cases

↓

Instructor reviews spreadsheet
Final decision; 2+ weeks

Legend

- Student action
- Staff action
- High-friction step
- Pain point
- Instructor action

Component B Deliverables

Three Cursor Mode

Type	Prompt	Output	What it did
Adding 1 feature	You are working on the Wayfinder app. for each of the resource, add a made up map to plot the location of the resource.	Adds a map UI and circle to pinpoint where the resources are located on a map	Made up Latitude and longitude coordinates near GIX for each of the resource. Creates a one row pandas DataFrame, creates a map for each of the resource

Information Hierarchy Review

Squint Test:

- Heading, Navbar on the left, map ui next to each resource

Question	Your Answer
What is the most important piece of information on the page?	The resources that the user search up
Is that information the most visually prominent element?	**Yes** / No
Can a first-time user understand what this app does at a glance?	**Yes** / No
Are related items visually grouped together?	**Yes** / No
Is there any element that draws attention but is not important?	The map UI is probably not as important at a first

	time glance, it should be hidden until users click on a specific resource.

Type	Prompt	Output	What it did
Revising Hierarchy	hide the map uis until the user clicks on a specific resource	Hides map ui until user checks show location on map	map uis are not created until user clicks to show map

- I revised the map so that it is only visible when toggled by the user, since the maps are not immediately important to see, the attention should first be on the resources.

Prompt Engineering

Type	Prompt	Output	What it did
Vague	Make the app look better	Add color scheme, updates visual for each of the color cards and the result heading showing how many matches	Updates the CSS, makes the resources cards more narrow and centered in the middle of the page, added a purple color scheme to the UI
Specific	Have the result heading appear at the top of the screen and take the whole width of the page excluding the space taken by the navbar. The heading should display the number of matching results in the format of Search Result: #.	The result appears at the top of the screen and takes the whole width of the page. The result # successfully shows how many matching resources there are and are in the right color.	Updates CSS, changes the location of the search results display, updates the colors of the text

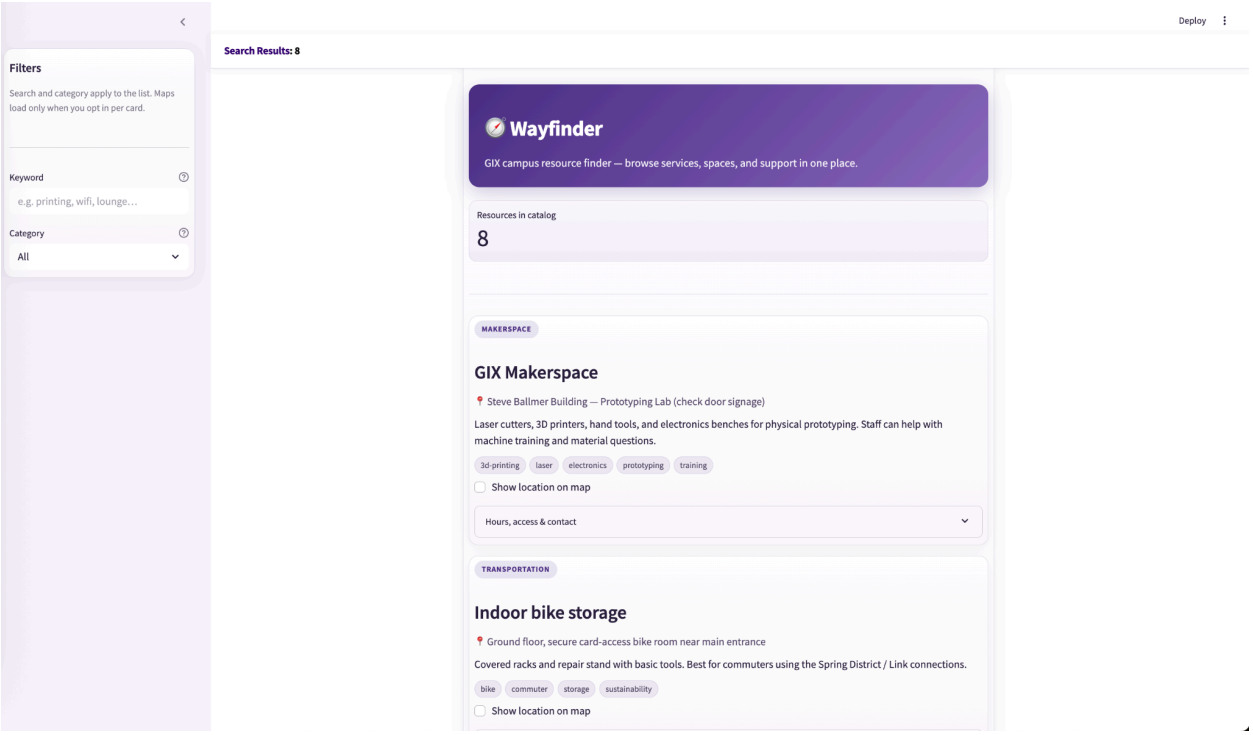
	“Result” should be a 32006e color and the number should be black. When the user searches up keywords in the search bar, the number of matched results should be accurately shown in the heading.		
Constrained	Add a full-width search results heading to the existing search page. Placement & Layout Position the heading in a fixed bar directly below the navbar, spanning 100% of the remaining viewport width Use calc(100vw - [navbar-width]) if the navbar is vertical, or width: 100% if horizontal — match whichever layout the project already uses Do not shift or overlap any existing content; push the results list down by the height of this bar Content & Format Display the text: Search Results: [n] where [n] is the count of currently	clears up the clutter in the header section, search result and number displayed as a fixed navbar on top	creates a sticky navbar on top of the screen showing “Result: #”, text size was decreased, color remains the same as the specific prompt

	visible/matched results "Search Results" → color #32006e; [n] → color #000000 (black); both in the same font-size and weight already used for page headings in this project Behavior On every keystroke in the existing search bar, recount matched results and update [n] in real time — no debounce delay unless one already exists in the project's search logic If the search bar is empty, [n] = total number of results (not 0) If the query matches 0 results, display Search Results: 0 — do not hide the heading If the results data hasn't loaded yet, display Search Results: — (em dash) as a loading placeholder Error handling If the results array is null or undefined, treat it as 0 and log a console warning: [SearchHeading] results data was null		

	or undefined Do not throw or crash the page; degrade gracefully Conventions Follow the component structure, naming conventions, and styling approach already used in this codebase (e.g. if the project uses Tailwind, use Tailwind; if it uses CSS modules, use CSS modules) The heading should be its own isolated component so it can be tested independently		

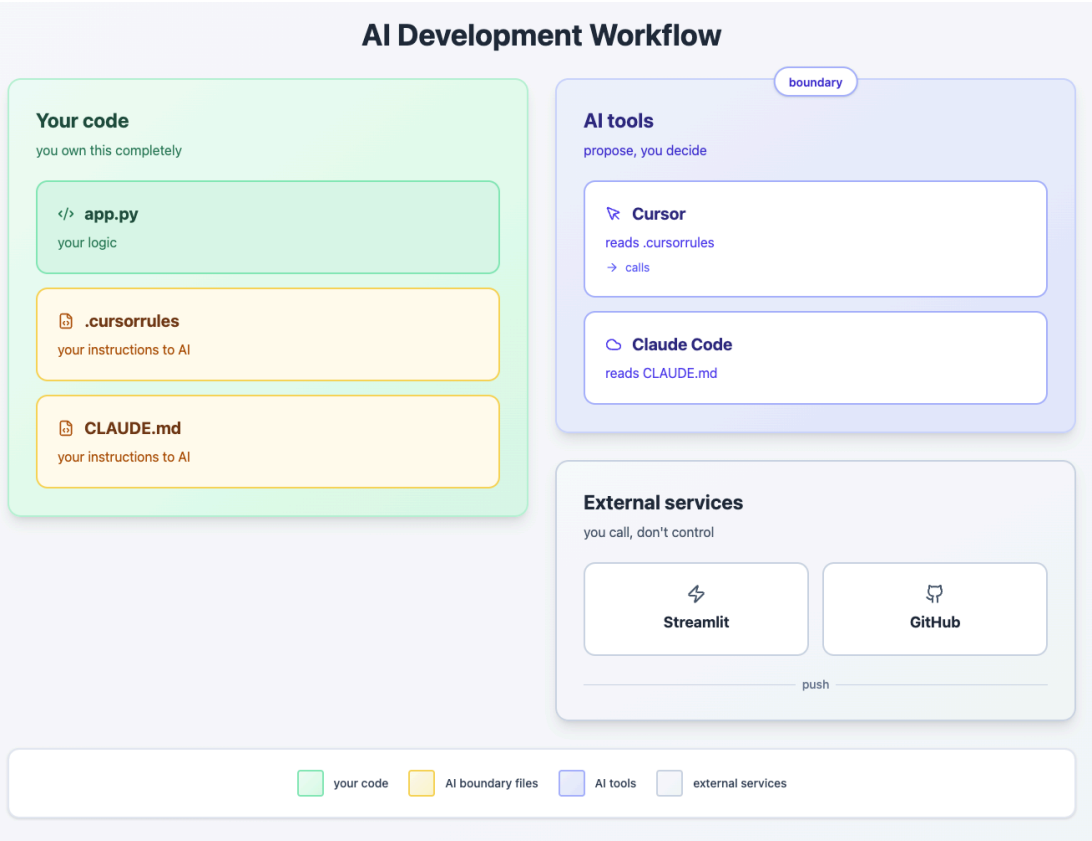
1. What was the biggest difference between your vague and specific prompts?
    a. The vague prompts left a lot of the changes up to the AI, and sometimes it would change things or add features that you did not want. The specific prompt leaves less guesswork to the AI by telling it exactly what you want.
2. What kind of detail mattered most -- layout, data, styling, or behavior?
    a. I think the styling mattered the most since there is the most room for interpretation without much guidance. For behavior, the AI can usually assume what normally an app would do if you provide it a brief overview of your product.
3. How much of the final result came from your instructions versus the AI's assumptions?
    a. For the vague prompts, almost all of it was from AI's instruction, however the very specific prompts allow almost no guesswork from the AI.

1. Screenshot of your enhanced app running



Component C Deliverables

1. Architecture diagram



2. Design Decision Log

Decision: Stable project-wide defaults go into .cursorrules and claude.md, while task-specific instructions go in the prompt. Try not to duplicate policy across three places.

Alternatives considered: Prompts only; rules only; mirror everything everywhere; single mega-file for all tools.

Why you chose this: Different tools read different files. Rules give persistent context without retyping. Prompts handle one-off goals and overrides. Git keeps rules versioned and shared.

Trade-off Rules can drift if maintained separately. Prompts alone miss standing conventions. Over-stuffed rules add noise to every future session.

When would you choose differently? Throwaway experiments → prompts only. Single tool, single file → skip the split. Strict compliance needs → lean hard on checked-in rules, minimize prompt overrides.

Think about bullets

Rules vs retyping into every prompt: Rules apply automatically and consistently. Prompts handle what's needed right now

Plotly vs Matplotlib conflict: The prompt usually wins, but it's not guaranteed. Best approach: don't let them conflict. If overriding, say so explicitly.

What belongs where: Rules: stack, lint, naming, security boundaries. Prompts: scope, edge cases, one-off tradeoffs.

Cost of many places: Every policy change needs N edits and risks contradiction. Fix with a single source and thin pointers elsewhere.

Component D Deliverables

1. Validation results (completed validation evidence)

#	Prompt Used	Without Config (Before)	With Config (After)	What Changed	Better/Worse/Same
1	In Lab 1/Wayfinder/app.py, add a sidebar selectbox to sort the filtered results by name (A–Z / Z–A) before the loop that renders cards. Match existing Streamlit patterns and keep type hints.	Suggested sorting inline in the loop with minimal typing; sometimes omitted return types or put logic in main() without a small helper.	Sorting aligned with sidebar filters; used list[dict[str, object]] or similar to match data.py; mentioned running Wayfinder from Lab 1/Wayfinder/.	Type hints and structure closer to repo; optional mention of smoke-testing Wayfinder from the correct folder.	*Better*

2	Explain what search_heading.py does and how it interacts with app.py — especially the fixed heading bar and when the script reruns.	Generic Streamlit explanation of st.markdown and CSS, little tie-in to "Wayfinder" or file layout.	Named Wayfinder, sidebar vs fixed bar, session-state CSS guard, and rerun behavior consistent with your code/comments.	Used project structure and app-specific behavior from context/rules.	*Better*
3	In Wayfinder, if a resource has contact: None, make sure the UI never crashes and still shows a sensible line — follow the same patterns as the rest of the app.	Broad try/except or string concat that could still edge-case; might not mention st.warning or logging.	Specific handling, user-visible st.info/st.caption-style messaging, optional logging, no bare except — aligned with CLAUDE.md / lab1-streamlit.mdc.	Error handling and Streamlit messaging matched documented conventions.	*Better*

2. Quality gate checklist (completed)

Before you submit, every item below must be satisfied:

- [x] Smoke test completed: For each prompt output, you confirmed (1) it runs, (2) the core behavior works once, and (3) invalid input is handled without crashing
- [x] 3 prompts tested: Your recording template has 3 complete rows with before/after observations

- [x] Differences are specific: Each "What Changed" entry describes a concrete, observable change (e.g., "added type hints" not "output improved")
- [x] Config files restored: Your `.cursorrules` and `CLAUDE.md` files are back in place and working (run one prompt to verify)
- [x] Honest assessment: If a config file made no difference on a particular prompt, you documented that honestly (marking "Same")
- [x] Prompts are documented: The exact prompt text is recorded so someone else could reproduce the test

Component E Deliverables (In Lab 2 github)

1. 5-sentence specification document for the eligibility checker
    a. SPEC.md in career-eligibility folder
2. Two implementations generated by two different AI coding tools (saved as separate files)
    a. eligibility-nextjs (cursor)
    b. eligibility-vanilla (
3. Project-level AI configuration file (`.cursorrules` or equivalent)
    a. .cursorrules in eligibility-nextjs
    b. CLAUDE.md in eligibility-vite-react
4. Comparison table evaluating both implementations
    a. in COMPARISON_AND_TESTING.md
5. Decision flowchart for the eligibility logic (including at least 3 branches and a human review step)
    a. FLOWCHART.md
6. Smoke test evidence for each implementation (runs, core path works, invalid input handled without crash)
    a. Completed
7. Input testing evidence for each implementation (1 valid input and 1 invalid input, with results)
    a. Completed

Reflection Deliverables

Write 3-5 sentences addressing these questions. Include this in your Canvas submission.

1. **Tool comparison: When you tested the same prompt across different AI tools (the AI Fluency warm-up or Cursor vs. Claude Code during the lab), what differences did you notice? Which tool gave you better code, and which gave you a better explanation?**

I ran the same prompt in Cursor and Claude (online) and compared results during the AI fluency warmup. The main differences were file access, multi-step edits, and how much context each tool had by default.

For code, Cursor worked better since it applied changes directly to the right files and stayed aligned with the Wayfinder layout. Claude without repo access had to guess file paths and produced snippets I had to merge manually.

For explanations, Claude was stronger, it walked through logic more clearly and defined Streamlit rerun behavior in plain language without assuming my folder structure.

2. **Prompt quality: Look at your prompt log. What specifically changed between your vague first prompt and your constrained third prompt? What did you learn about what AI tools need from you to produce good results?**

My first prompt was vague, something like "make the app look better" with no file path, no acceptance criteria, no constraints. The model guessed and produced generic advice that didn't match my setup.

By my third constrained prompt I was specifying the exact file, the desired behavior, edge cases, conventions, and styling. The output was scoped, referenced the right modules, and was easier to review.

Key lesson: good prompts include goal, scope, files, definition of done, and project constraints, essentially leaving almost nothing up to AI interpretation. Vague intent forces improvisation.

3. **Configuration impact: After creating `.cursorrules` and `CLAUDE.md`, did you notice any change in the output you got from AI tools? How do configuration files change the way an AI coding agent behaves -- and what would you add to these files after today's experience?**

After adding CLAUDE.md and .cursorrules, AI output noticeably improved. With config present, responses correctly named Wayfinder vs ProcureGIX, suggested running Streamlit from the right folder, included type hints and html.escape, and avoided bare except. Without it, the model fell back to generic Python patterns and wrong paths.

Config files work by acting as persistent instructions injected alongside your prompt, they shift the model's defaults toward your stack and conventions instead of generic ones.

Going forward I'd add: preferred patterns for maps and metrics, how to run certain files and have a clear rule to not touch features that I am not currently working on.